

ALMA-13B-R under weight-only GPTQ: 16 / 8 / 4 / 3 / 2 bits on WMT'22

September 25, 2026

What is in here

ALMA-13B-R quantized by GPTQ (`GPTQModel 4.2.5`): weight-only, asymmetric, group size 128, act-order, every Linear of the 40 decoder layers quantized, embeddings / `lm_head` / norms / scales left in fp16. Generation is beam 5, bf16, seed 42, the protocol of `ours-beam` (the fp16 reproduction, which is the baseline here), scored on WMT'22 for de, cs, zh, ru and WMT'21 for is. Table 1 is the grid, Table 2 the cost against fp16, Table 3 the checkpoint sizes; per-direction numbers are in the appendix tables, and the raw `outputs/baseline/<run>.tsv` files stay the machine-readable copy.

The grid

	BLEU	chrF++	COMET-22	KIWI-22	KIWI-XXL	XCOMET-XXL	MetricX-24	Halluc.	GiB	ratio
<i>en→xx (5 directions)</i>										
W2G128	0.03	3.41	29.74	23.56	-2.12	32.27	10.40	47.43	3.8	6.41×
W3G128	20.91	41.22	82.02	77.48	71.15	83.37	4.09	4.01	5.3	4.60×
W4G128	25.71	46.81	87.71	83.45	85.87	93.79	1.95	0.81	6.8	3.59×
W8G128	26.97	47.36	88.04	83.69	86.77	94.64	1.88	0.58	12.7	1.91×
FP16	26.98	47.39	88.07	83.70	86.79	94.55	1.87	0.57	24.2	1.00×
FP16 (sampling)	26.89	47.40	88.07	83.74	86.78	94.67	1.87	0.54	24.2	1.00×
<i>xx→en (5 directions)</i>										
W2G128	0.04	3.95	24.35	29.16	-0.14	13.05	9.42	49.83	3.8	6.41×
W3G128	25.63	50.70	78.39	75.27	68.12	73.34	5.34	0.74	5.3	4.60×
W4G128	35.24	59.73	84.93	81.24	82.16	88.76	3.02	0.07	6.8	3.59×
W8G128	35.83	60.16	85.29	81.51	82.78	89.59	2.88	0.16	12.7	1.91×
FP16	35.82	60.14	85.29	81.50	82.74	89.57	2.88	0.16	24.2	1.00×
FP16 (sampling)	35.85	60.13	85.28	81.51	82.76	89.58	2.88	0.15	24.2	1.00×
<i>ALMA-R paper (reported, their decoding)</i>										
avg en→xx	27.03	–	87.74	83.34	85.74	94.05	–	–		
avg xx→en	35.45	–	85.21	81.33	82.43	89.11	–	–		

Table 1: ALMA-13B-R on WMT’22 (WMT’21 for **is**): weight-only GPTQ, asymmetric, group size 128, act-order, every decoder Linear quantized, beam 5, bf16, seed 42. FP16 is our beam-search reproduction (**ours-beam**); **FP16 (sampling)** is the paper’s beam-sampling decoding and is *not* the reference for the quantized rows. MetricX-24 and the hallucination rate (% of sentences at least twice the reference length) are the two where lower is better. GiB is the checkpoint on disk, ratio the saving against FP16.

	BLEU	chrF++	COMET-22	KIWI-22	KIWI-XXL	XCOMET-XXL	MetricX-24	Halluc.
<i>en→xx</i>								
W2G128	-26.94	-43.98	-58.33	-60.14	-88.91	-62.29	+8.53	+46.86
W3G128	-6.06	-6.17	-6.05	-6.22	-15.64	-11.18	+2.22	+3.44
W4G128	-1.27	-0.57	-0.36	-0.24	-0.92	-0.76	+0.08	+0.23
W8G128	-0.01	-0.03	-0.03	-0.01	-0.02	+0.09	0.00	+0.01
<i>xx→en</i>								
W2G128	-35.78	-56.18	-60.94	-52.34	-82.88	-76.53	+6.53	+49.67
W3G128	-10.19	-9.43	-6.90	-6.23	-14.62	-16.23	+2.46	+0.58
W4G128	-0.58	-0.41	-0.36	-0.26	-0.58	-0.81	+0.13	-0.09
W8G128	0.00	+0.02	0.00	+0.01	+0.04	+0.02	0.00	0.00

Table 2: Difference against the FP16 beam baseline (Table 1): green is better than FP16, red is worse, saturation scaled to the largest change in that column. These are the numbers that say what the compression costs.

precision	bits	GiB	GB	vs FP16
W2G128	2	3.78	4.06	6.41×
W3G128	3	5.27	5.66	4.60×
W4G128	4	6.76	7.26	3.59×
W8G128	8	12.72	13.65	1.91×
FP16	16	24.24	26.03	–

Table 3: Checkpoint size on disk. Only the 40 decoder layers’ Linears are quantized; the embeddings, **lm_head**, the norms and the scales and zero-points stay fp16, a fixed ≈ 0.8 GiB floor. That is why the end-to-end ratio stays well under $16/b$ even at 2 bits (**quantization_sizes.txt**).

precision	bits	checkpoint		de-en, src 256, BATCH=4						zh-en, src 512, BATCH=1						
		kernel	GiB on disk	load	peak	GiB	peak	GiB	fixed s	real lines		peak	GiB	fixed s	real lines	
										s	n				s	n
FP16	16	fp16	24.24		24.3		28.8	0.359	0.436	1984		26.5	0.517	1.471	256	
W8G128	8	TritonV2	12.72		13.0		17.2	0.607	0.755	512		15.1	0.877	2.683	256	
W4G128	4	ExLLamaV2	6.76		7.7		12.3	0.460	0.550	512		10.0	0.543	1.534	256	
W3G128	3	Torch	5.27		5.6		10.1	1.181	1.467	64		7.8	1.377	4.570	64	
W2G128	2	TritonV2	3.78		4.1		11.7	4.705	3.513	64		6.4	10.138	10.577	64	

Table 4: Measured cost per checkpoint, one H100, one process per row (`francesco/analysis/sbatch/cost.sbatch`, `measure_quant_cost.py`). *load peak* is the allocator peak while the checkpoint was read (for weight-only PTQ the packed tensors are what is loaded, and on disk they are already exactly that size); *peak* is the allocator peak of the timed `generate()` calls (reserved, i.e. fragmentation included, is in the JSON), one warmup excluded. *fixed s* is the median of three calls on the first four sentences of the pair - a byte-identical workload for every row, 30 generated tokens; *real lines* is the same model over *n* full lines of the pair at the same batch, which is the number comparable to a grid log. The two protocols are the ones the eval jobs run: de-en 256 at BATCH=4 (20 sequences per forward) and zh-en 512 at BATCH_LONG=1 (5). The fp16 row loads through transformers, the packed checkpoints through GPTQModel; prompts, padding, bf16, beam 5 and seed 42 are identical.

What the quantization costs and buys

The quality side of this grid is Tables 1–2. This section is the other half of the same decision, measured on one H100 with the eval protocol’s own shapes (`francesco/analysis/measure_quant_cost.py`, `sbatch/cost.sbatch` for the checkpoints, `sbatch/probe16.sbatch` for the batch sweep, plus each eval job’s own log for the real grid times): what a checkpoint weighs, what it needs in VRAM, how fast it generates and what the grid paid in GPU-hours.

On this stack the compression buys *memory*, not time. Peak VRAM at the eval protocol falls 28.8 GiB (fp16) → 17.2 (w8) → 12.3 (w4) on the de-en shape, and per line nothing gets faster: 0.359s (fp16) → 0.603 (w8) → 0.461 (w4) → 1.188 (w3) → 4.709 (w2) on a byte-identical workload. The reason is the kernel rather than the bit width: GPTQModel’s Triton and Torch linears dequantize the whole weight matrix to bf16 and then matmul, so a forward pass reads the packed weights *and* writes and re-reads a full bf16 copy of them — more HBM traffic than fp16, which reads its weights once. w4 is the exception that proves the rule: it is the one checkpoint that gets `ExllamaV2QuantLinear`, and it is the only quantized row within $1.3\times$ of fp16 per line. The spread across bit widths is the kernel lottery, not a smooth function of precision.

The memory saving is also smaller than the disk saving, because the KV cache and the activations do not shrink with the weights: at fp16 the weights are 84% of the de-en BATCH=4 peak, at w4 55%, so each further step below 4 bits buys 1–2 GiB of peak rather than the 3 GiB the checkpoint loses — while the quality cost of those steps rises from 0.6 XCOMET-XXL (w4) to 10.9 (w3) to 53.8 (w2). That is the shape of the answer: w8 is free, w4 is cheap, w3 and w2 are not worth keeping on cost grounds alone, and the case for the grid is the low-resource-language experiment it was built to run, not throughput.

Which kernel GPTQModel auto-selected is the `kernel` column of Table 4; Table 8 is the sweep over every kernel it will load for these checkpoints, including the ones that refuse them.

direction	FP16	W8G128	W4G128	W3G128	W2G128
de→en	0.284	0.750	0.541	1.382	3.584
cs→en	0.318	0.870	0.622	1.558	2.764
is→en	0.303	0.876	0.630	1.914	2.766
zh→en	1.471	3.069	1.757	5.901	9.770
ru→en	0.309	0.812	0.583	1.512	3.330
en→de	0.412	1.102	0.784	1.953	3.688
en→cs	0.506	1.343	0.969	2.739	3.461
en→is	0.797	2.124	1.524	5.598	3.714
en→zh	0.594	1.591	1.161	3.582	3.558
en→ru	0.470	1.258	0.901	2.683	3.243
all 10	0.546	1.369	0.937	2.801	4.066
<i>GPU-h charged</i>	10.6	6.6	4.5	13.6	19.7
<i>GPU-s/line</i>	2.18	1.37	0.94	2.80	4.07
<i>vs FP16</i>	1.00 <i>imes</i>	0.63 <i>imes</i>	0.43 <i>imes</i>	1.28 <i>imes</i>	1.86 <i>imes</i>

Table 5: Seconds per input line for the whole WMT’22 grid, generation only, taken from each job’s own log (a pair’s wall divided by its lines). Every precision ran the same ten directions, the same beam-5 bf16 settings and the same sentences; only **zh-en** differs by run (source 512 at **BATCH_LONG=1** for the quantized runs, the same setting for fp16 - that pair comes from the cost probe, since the fp16 job’s **zh-en** attempt died out-of-memory). *GPU-h charged* is what the cluster billed for each run’s generation: the fp16 baseline is 4× its wall because that job ran four ranks over four GPUs with the test set sharded across them, every quantized run was one process on one GPU; *GPU-s/line* is that divided by the 17,471 lines, so it is the only row that is comparable across runs, and the per-line column above is a *latency* only for the single-GPU runs. Scoring is excluded.

precision	disk	peak	grid	quality Δ vs. FP16	measured on the grid			
					BLEU	chrF++	s/line	GPU-s/line
	GiB	GiB	GPU-h	XCOMET-XXL				
W8G128	12.72	17.2	6.6	+0.03	0.00	0.00	0.607 (1.69×)	1.37
W4G128	6.76	12.3	4.5	-0.60	-0.61	-0.35	0.460 (1.28×)	0.94
W3G128	5.27	10.1	13.6	-10.91	-6.61	-6.26	1.181 (3.29×)	2.80
W2G128	3.78	11.7	19.7	-53.84	-24.63	-39.09	4.705 (13.11×)	4.07

Table 6: What each checkpoint bought and cost. The Δ columns are the mean of the two 5-direction averages in **outputs/baseline/<run>.tsv** against the fp16 beam run, so they are the same numbers as Table 2, collapsed. *peak* is the de-en **BATCH=4** allocator peak from Table 4; *grid GPU-h* is the sum of the ten directions in that run’s own eval log (generation only, single GPU, scoring excluded - the w2 figure is the requeue’s, so it excludes the wall it lost); *GPU-s/line* is that divided by the 17,471 scored lines; *s/line* and its ratio are the probe’s de-en row, i.e. the same fixed-cost structure the full grid pays.

precision	s per line			peak GiB		
	B=4	B=16	gain	B=4	B=16	B=16 vs FP16
FP16	0.359	0.226	1.59×	28.8	42.3	1.00×
W8G128	0.603	0.266	2.27×	17.2	30.7	1.18×
W4G128	0.461	0.281	1.64×	12.3	26.0	1.24×
W3G128	1.188	0.427	2.78×	10.1	23.3	1.89×
W2G128	4.709	2.070	2.27×	11.7	35.4	9.16×

Table 7: The same fixed workload (de-en, source 256, beam 5, the first four sentences) at **BATCH=4** and **BATCH=16**, i.e. 20 and 80 sequences per forward, with the allocator peak of each. Every checkpoint gains from the batch and the low-bit ones gain most, because their per-forward dequantization is a fixed cost the batch amortizes - but the gain never closes the gap: at **BATCH=16** the quantized rows are still slower per line than fp16 at the same batch (*B=16 vs FP16*, above 1). What the compression does buy is the room to run that batch at all: w8 at **BATCH=16** needs 30.7 GiB and 0.266 s/line where fp16 at **BATCH=4** needs 28.8 GiB and 0.359 s/line, so spending the memory the quantization freed on a 4× larger batch is the one configuration where the compression converts into throughput.

checkpoint	kernel	de-en s/line	zh-en 512 @4 s/line	peak GiB
w2/Torch	torch	4.270	2.739	11.7
w2/TritonV2	triton	4.699	2.888	11.7
w3/Torch	torch	1.190	1.620	10.1
w4/ExLLamaV2	exllama_v2	0.462	0.728	12.3
w4/Torch	torch	0.561	0.777	11.3
w4/TritonV2	triton	0.618	0.831	11.3
w8/Torch	torch	0.549	0.790	17.2
w8/TritonV2	triton	0.601	0.834	17.2

Table 8: The same fixed workload (de-en 256 @ BATCH=4, and zh-en 512 at BATCH=4) on every kernel GPTQModel 4.2.5 will load for these checkpoints. Three things came out of the sweep: **ExllamaV2QuantLinear** is 1.34× faster than Triton for 4 bits, **TorchQuantLinear** is 9 % faster than Triton for 8 (Torch’s `dequantize_weight` is a 22ms pass versus the Triton kernel’s 2.4s cold), and **MarlinQuantLinear** - the only fused kernel that declares 8-bit support - refuses these checkpoints because it requires *symmetric* weights (`only supports [True] bits: actual sym = False`). `exllama_v1` is not installed and BitBLAS needs `desc_act=False`. No kernel in the stack gives 2- or 3-bit a fused path.

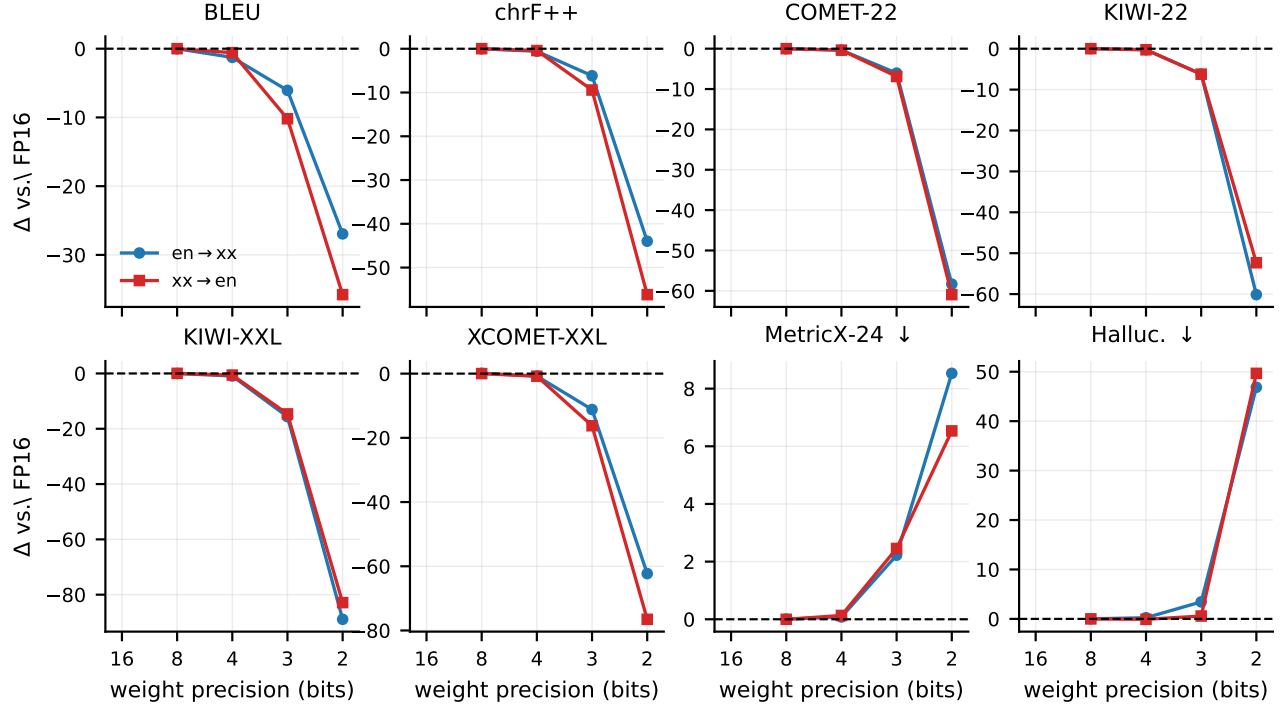


Figure 1: Every metric against the FP16 beam baseline, per translation direction. The x axis is the weight precision, one tick per quantized checkpoint; the dashed line is FP16 itself, so a point below it (above, for the two marked ↓) is a loss. A precision whose run has no scores yet is simply absent from the axis.

Figures

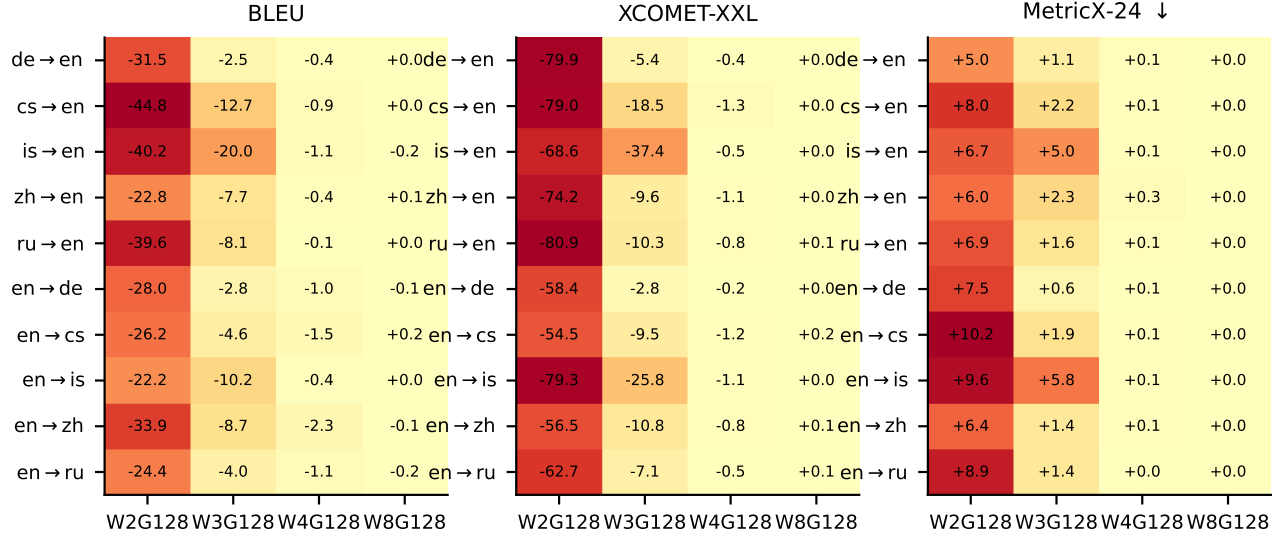


Figure 2: Difference against FP16 per direction and precision, coloured so that red is worse and green is better. MetricX-24 is sign-flipped for the colour only: the printed number is the real Δ , and its sign is the opposite of the colour. Each panel is scaled to its own worst cell, so the panels are comparable in shape, not in magnitude.

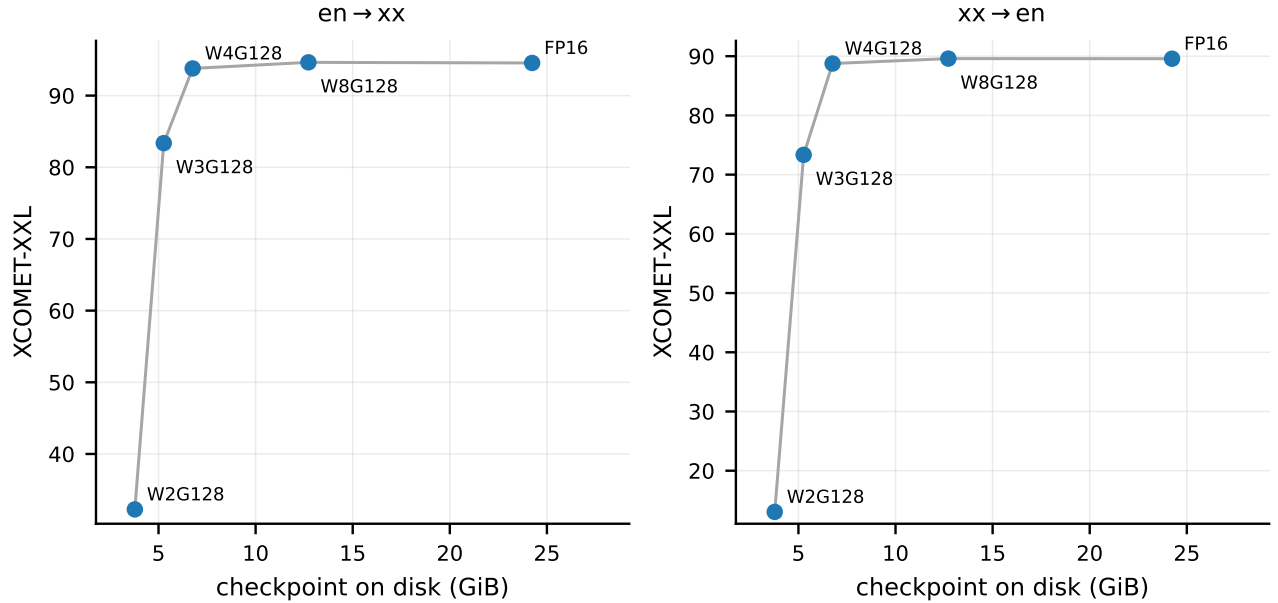


Figure 3: XCOMET-XXL against the checkpoint size on disk for both halves of the grid. The 8-bit checkpoint is 1.9× smaller and indistinguishable from FP16; 4 bits costs well under a point for another 1.9×; the 3-bit step is where both the size and the score move.

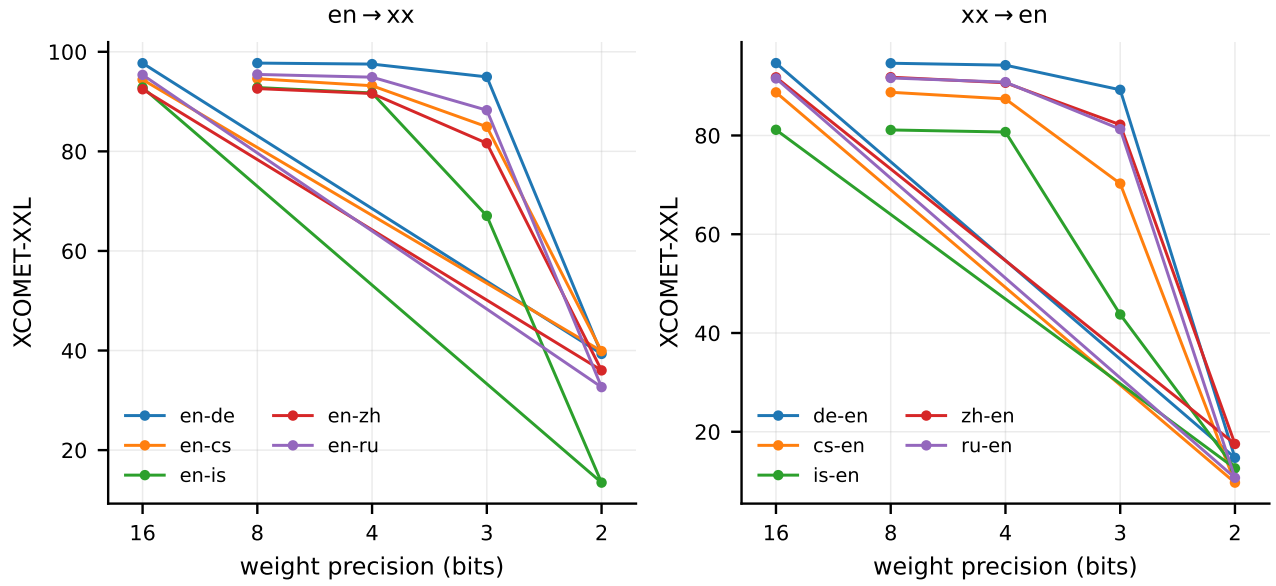


Figure 4: XCOMET-XXL per direction across the precision grid. All ten directions are hit, but not evenly: the low-resource ones (is, zh, cs) lose several times what de or ru lose at the same precision.

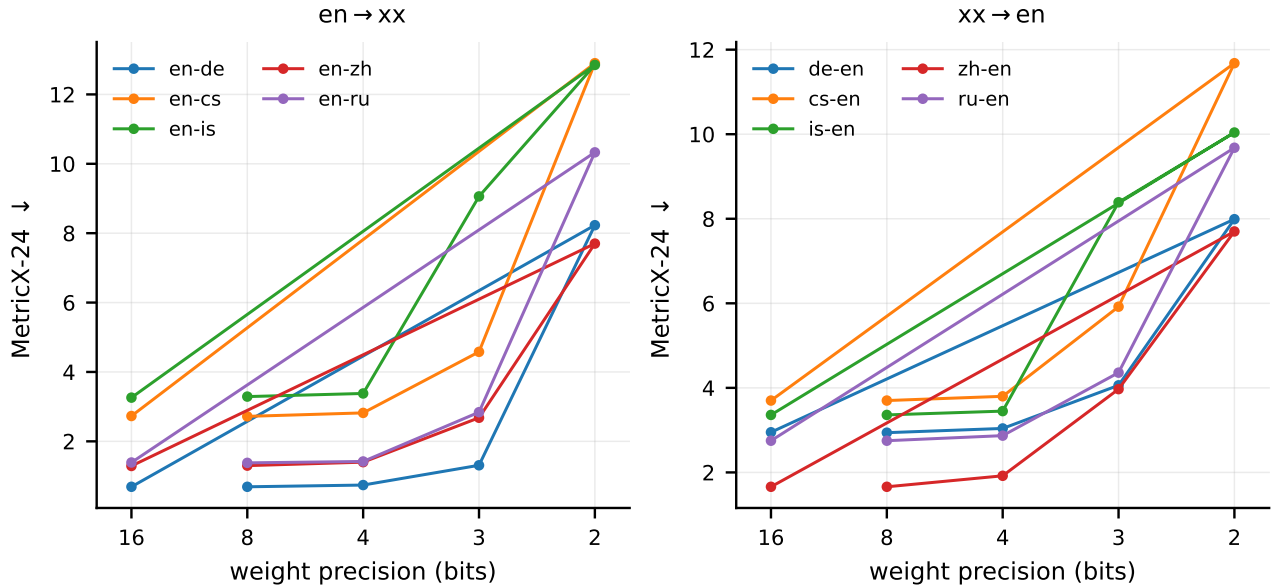


Figure 5: MetricX-24 per direction across the precision grid. All ten directions are hit, but not evenly: the low-resource ones (is, zh, cs) lose several times what de or ru lose at the same precision.

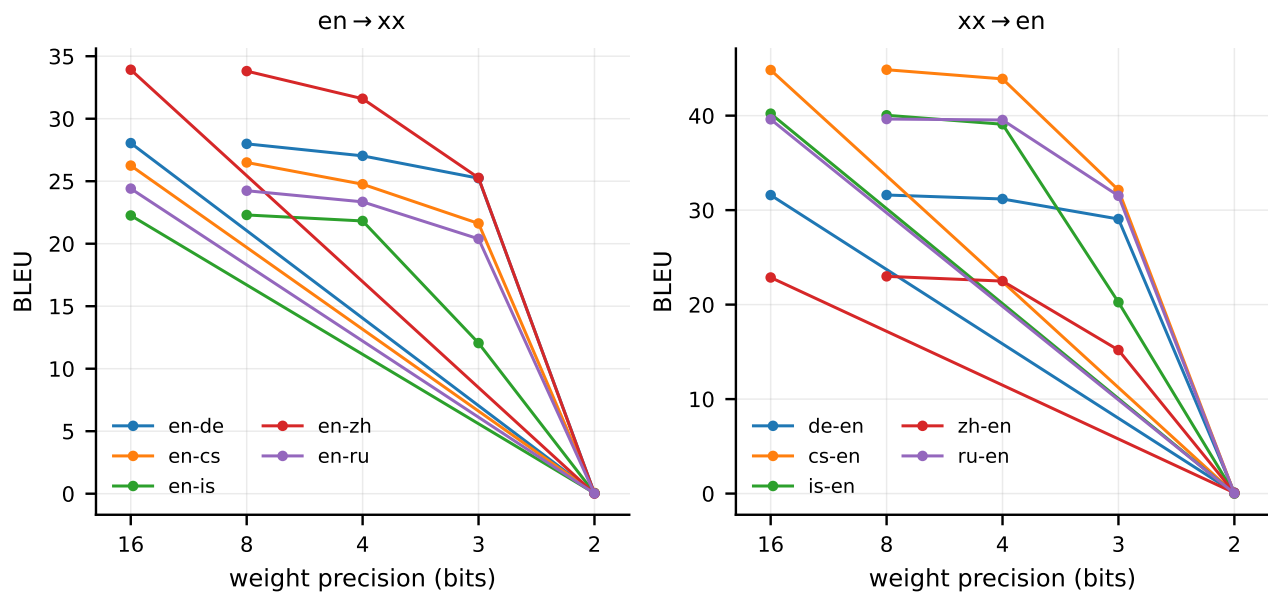


Figure 6: BLEU per direction across the precision grid. All ten directions are hit, but not evenly: the low-resource ones (is, zh, cs) lose several times what de or ru lose at the same precision.

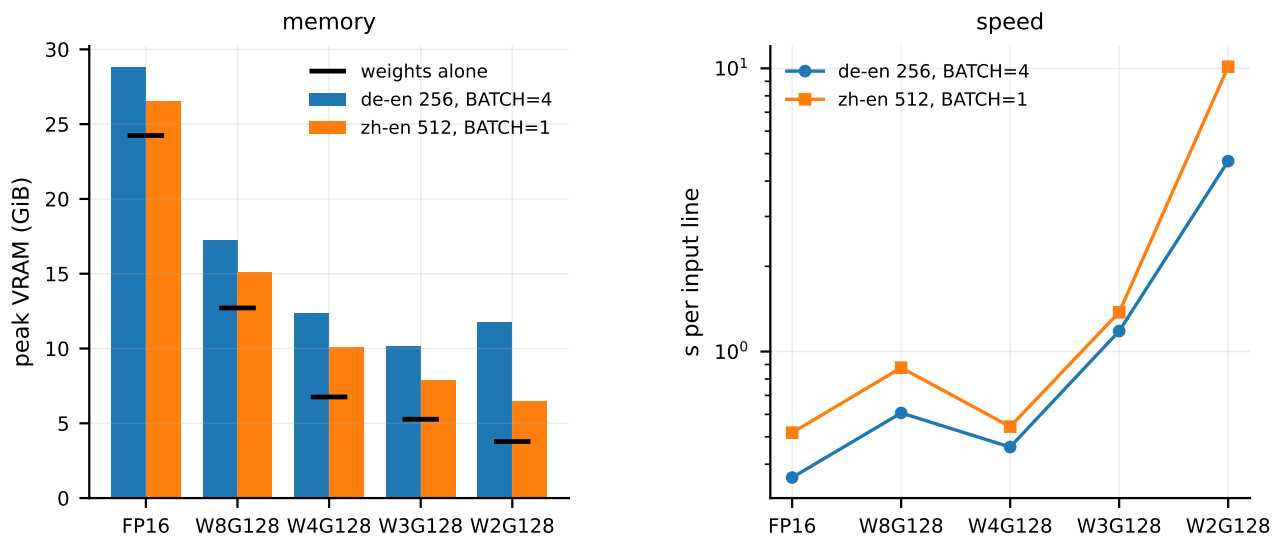


Figure 7: Left: the allocator peak of a timed `generate()` at the two eval protocols, with the model’s own parameter footprint marked. The weights are what the compression shrinks; the rest of the bar is the KV cache and the activations, which no weight-only scheme touches, so the saving saturates well before it reaches the weights’ ratio - and w2’s peak rises again over w4’s, because it generates 180-token outputs where w4 generates 40 and the extra KV cache outweighs the smaller weights. Right: the same checkpoints’ cost per input line, log scale. The low-bit checkpoints are not cheaper per line than fp16 on this stack - GPTQModel’s Triton/Torch kernels dequantize the weights and then matmul (`quant.matmul` in `triton_utils/dequant.py`), so every forward pays for weights it never stores - unless the batch is raised (see the batch-size section).

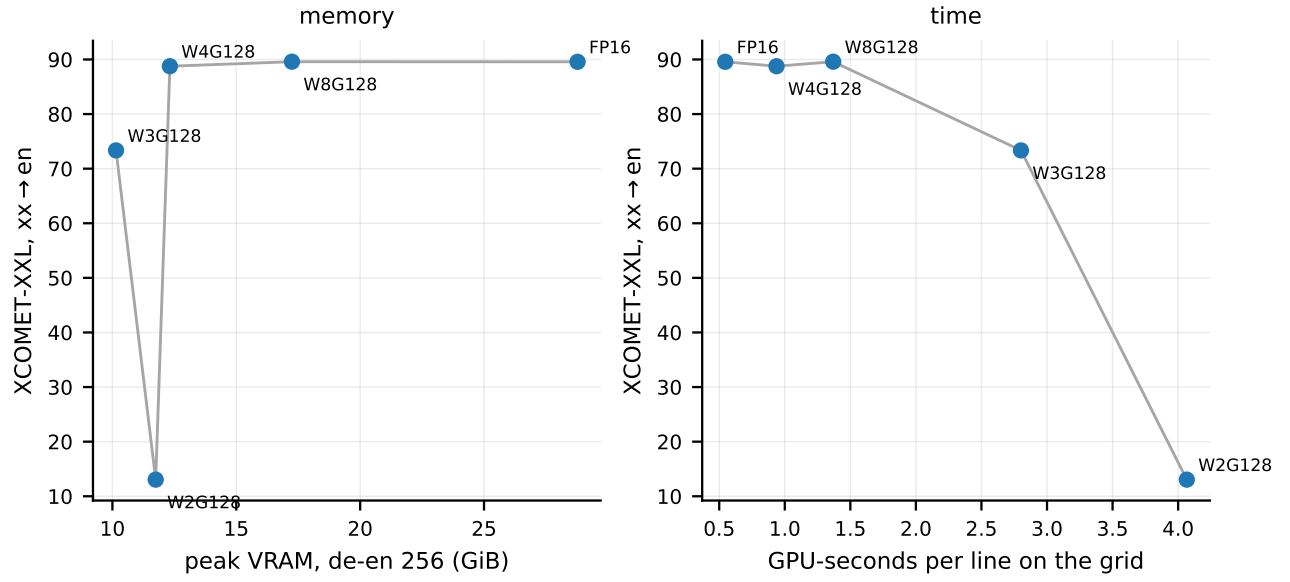


Figure 8: The trade the grid made. XCOMET-XXL is the harness the 3-bit collapse shows up in first (81.15 to 43.75 on `is-en`), so it is the metric that separates the two checkpoints that are worth keeping from the two that are not. Memory is the de-en `BATCH=4` peak; time is the measured grid (generation only), GPU-seconds per scored line.

	W2G128	W3G128	W4G128	W8G128	FP16	FP16 (sampling)	paper
de→en	0.10	29.06	31.18	31.60	31.59	31.59	30.89
cs→en	0.01	32.13	43.89	44.86	44.83	44.85	44.39
is→en	0.00	20.25	39.09	40.04	40.21	40.15	39.67
zh→en	0.08	15.19	22.49	22.99	22.87	22.99	23.23
ru→en	0.02	31.52	39.54	39.64	39.61	39.65	39.06
en→de	0.04	25.24	27.03	27.99	28.05	27.88	27.72
en→cs	0.06	21.62	24.76	26.50	26.25	26.16	26.32
en→is	0.04	12.05	21.82	22.30	22.26	22.17	22.88
en→zh	0.01	25.27	31.60	33.81	33.92	33.89	34.06
en→ru	0.02	20.39	23.35	24.24	24.41	24.36	24.15
avg_en→xx	0.03	20.91	25.71	26.97	26.98	26.89	27.03
avg_xx→en	0.04	25.63	35.24	35.83	35.82	35.85	35.45

Table 9: BLEU by direction, with the ALMA-R paper’s own reported numbers.

	W2G128	W3G128	W4G128	W8G128	FP16	FP16 (sampling)
de→en	4.71	52.80	55.03	55.27	55.27	55.25
cs→en	3.47	56.68	66.52	67.35	67.33	67.30
is→en	4.37	43.15	61.13	61.69	61.72	61.65
zh→en	3.83	42.78	51.79	52.26	52.20	52.24
ru→en	3.39	58.11	64.16	64.21	64.17	64.21
en→de	6.45	52.50	54.95	55.63	55.66	55.53
en→cs	4.65	47.59	51.62	52.36	52.25	52.30
en→is	5.09	38.50	50.32	50.56	50.58	50.52
en→zh	0.36	21.98	27.46	28.23	28.37	28.53
en→ru	0.51	45.54	49.72	50.01	50.08	50.13
avg_en→xx	3.41	41.22	46.81	47.36	47.39	47.40
avg_xx→en	3.95	50.70	59.73	60.16	60.14	60.13

Table 10: chrF++ by direction.

Per-direction numbers

	W2G128	W3G128	W4G128	W8G128	FP16	FP16 (sampling)	paper
de→en	26.51	82.32	84.76	85.02	85.02	84.99	84.95
cs→en	23.77	80.76	86.67	87.00	87.00	86.99	86.85
is→en	23.63	73.08	86.83	87.25	87.26	87.24	87.14
zh→en	24.94	75.17	81.07	81.61	81.58	81.60	81.64
ru→en	22.90	80.63	85.31	85.58	85.59	85.60	85.45
en→de	34.22	84.07	86.47	86.85	86.93	86.88	86.40
en→cs	34.15	86.32	90.27	90.56	90.60	90.52	90.29
en→is	20.92	74.00	86.77	87.03	87.05	87.12	86.85
en→zh	33.89	81.24	86.53	87.04	87.03	87.05	86.86
en→ru	25.51	84.46	88.50	88.73	88.75	88.78	88.30
avg.en→xx	29.74	82.02	87.71	88.04	88.07	88.07	87.74
avg.xx→en	24.35	78.39	84.93	85.29	85.29	85.28	85.21

Table 11: COMET-22 by direction, with the ALMA-R paper’s own reported numbers.

	W2G128	W3G128	W4G128	W8G128	FP16	FP16 (sampling)	paper
de→en	28.14	79.01	81.50	81.69	81.69	81.69	81.50
cs→en	26.51	76.66	82.69	82.79	82.78	82.76	82.63
is→en	34.41	70.49	81.54	81.66	81.68	81.67	81.57
zh→en	27.44	72.75	78.86	79.51	79.46	79.54	79.24
ru→en	29.29	77.46	81.60	81.90	81.89	81.88	81.72
en→de	23.93	80.98	83.40	83.69	83.69	83.68	83.28
en→cs	23.83	80.41	85.17	85.34	85.37	85.40	84.99
en→is	21.25	69.83	82.14	82.39	82.44	82.58	82.18
en→zh	24.61	76.35	82.31	82.59	82.58	82.61	82.25
en→ru	24.18	79.83	84.24	84.43	84.40	84.45	83.98
avg.en→xx	23.56	77.48	83.45	83.69	83.70	83.74	83.34
avg.xx→en	29.16	75.27	81.24	81.51	81.50	81.51	81.33

Table 12: KIWI-22 by direction, with the ALMA-R paper’s own reported numbers.

	W2G128	W3G128	W4G128	W8G128	FP16	FP16 (sampling)	paper
de→en	0.35	77.68	84.04	84.48	84.45	84.43	83.97
cs→en	-1.38	68.33	83.56	84.26	84.22	84.18	83.75
is→en	-1.02	58.13	85.53	85.95	85.97	85.93	85.73
zh→en	2.24	64.31	76.49	77.51	77.37	77.50	77.17
ru→en	-0.87	72.13	81.16	81.70	81.69	81.76	81.54
en→de	-2.09	77.76	84.46	85.31	85.30	85.25	84.25
en→cs	-1.34	75.17	87.55	88.74	88.74	88.53	87.06
en→is	-3.68	56.31	85.58	86.30	86.41	86.58	85.68
en→zh	-1.62	70.56	84.26	85.14	85.23	85.25	84.32
en→ru	-1.88	75.97	87.51	88.36	88.28	88.27	87.37
avg.en→xx	-2.12	71.15	85.87	86.77	86.79	86.78	85.74
avg.xx→en	-0.14	68.12	82.16	82.78	82.74	82.76	82.43

Table 13: KIWI-XXL by direction, with the ALMA-R paper’s own reported numbers.

	W2G128	W3G128	W4G128	W8G128	FP16	FP16 (sampling)	paper
de→en	14.72	89.25	94.24	94.63	94.66	94.67	94.20
cs→en	9.70	70.26	87.40	88.75	88.73	88.63	88.03
is→en	12.60	43.75	80.70	81.12	81.15	81.12	80.49
zh→en	17.53	82.18	90.67	91.81	91.78	91.91	91.65
ru→en	10.68	81.28	90.80	91.65	91.55	91.55	91.18
en→de	39.32	94.96	97.53	97.73	97.71	97.70	97.48
en→cs	39.88	84.94	93.18	94.59	94.41	94.42	93.61
en→is	13.47	67.05	91.73	92.82	92.82	93.27	91.93
en→zh	36.01	81.63	91.62	92.61	92.47	92.61	92.03
en→ru	32.65	88.27	94.90	95.45	95.36	95.37	95.22
avg_en→xx	32.27	83.37	93.79	94.64	94.55	94.67	94.05
avg_xx→en	13.05	73.34	88.76	89.59	89.57	89.58	89.11

Table 14: XCOMET-XXL by direction, with the ALMA-R paper’s own reported numbers.

	W2G128	W3G128	W4G128	W8G128	FP16	FP16 (sampling)
de→en	7.99	4.06	3.04	2.94	2.95	2.95
cs→en	11.68	5.92	3.80	3.70	3.70	3.70
is→en	10.04	8.39	3.45	3.36	3.36	3.36
zh→en	7.70	3.97	1.92	1.66	1.66	1.65
ru→en	9.68	4.36	2.87	2.75	2.75	2.75
en→de	8.23	1.31	0.74	0.69	0.69	0.68
en→cs	12.90	4.58	2.82	2.72	2.73	2.72
en→is	12.85	9.06	3.38	3.29	3.26	3.24
en→zh	7.70	2.68	1.40	1.30	1.29	1.30
en→ru	10.33	2.84	1.42	1.38	1.39	1.40
avg_en→xx	10.40	4.09	1.95	1.88	1.87	1.87
avg_xx→en	9.42	5.34	3.02	2.88	2.88	2.88

Table 15: MetricX-24 by direction.

	W2G128	W3G128	W4G128	W8G128	FP16	FP16 (sampling)
de→en	62.05	0.10	0.00	0.05	0.05	0.05
cs→en	46.27	0.21	0.00	0.07	0.07	0.07
is→en	42.70	0.80	0.00	0.00	0.00	0.00
zh→en	40.80	2.29	0.37	0.59	0.53	0.48
ru→en	57.34	0.30	0.00	0.10	0.15	0.15
en→de	42.46	0.25	0.20	0.20	0.20	0.20
en→cs	31.81	2.31	0.44	0.39	0.49	0.49
en→is	39.40	9.90	0.10	0.20	0.20	0.00
en→zh	93.13	5.06	2.65	1.82	1.72	1.72
en→ru	30.34	2.55	0.64	0.29	0.25	0.29
avg_en→xx	47.43	4.01	0.81	0.58	0.57	0.54
avg_xx→en	49.83	0.74	0.07	0.16	0.16	0.15

Table 16: Halluc. by direction.